Project Plan

Quantum Routines: Implementation, Verification, and Resource Estimation

Kush Aggarwal - agg1810@uw.edu
Thomas Huang - yshuang@uw.edu
Christopher Sharp - sharpc42@uw.edu
Industry Mentor: Mariia Mykhailova (PsiQuantum)

Project Background

As the industry of quantum computing continues to develop, as hardware implementations are improved upon and finalized, there is attention being directed towards designing optimized algorithms and implementing classical routines on quantum computers so that companies, institutions, and universities can begin designing software for quantum computing without requiring the hardware to be finalized. This early and active software development will allow the goal of quantum computing to complement and improve on classical computing to be better realized. To do this, we must reformulate classical computing routines into quantum-friendly forms. In particular, the rules of quantum mechanics demand no cloning and no destruction of qubits. This means that as we map the number of classical bits to qubits, for the classical routine to be directly convertible to quantum we must also ensure that all bits in equals the number of bits out. Unitarity also demands a unique combination of output qubits per combination of input qubits. This all is the equivalent of saying that the routines must be reversible. Finding reversible computing routines that can be converted to quantum computing allows us to design these quantum versions of classical routines, for example basic mathematical operations like addition and multiplication.

Project Objective

We will develop reversible computing routines to implement in the Workbench, the Python library implemented in PsiQuantum's software development, circuit design, and resource analyzing tool Construct. Workbench is designed to allow writing of quantum programs in a familiar, Pythonic manner, e.g. accessing qubits in a register by slicing through the register like a list or numpy array. This allows more rapid implementation, iteration, and testing of a prospective quantum program or routine, and less room for design or implementation error from misunderstanding the API. Using it, we will each implement one of the algorithms from this paper (which explores implementations in Microsoft's Q#). These are to be implemented as "qubricks" in Workbench. We will then benchmark the resource utilization of our implementations using Construct to compare its performance to these previous implementations and try to optimize them as much as possible. Our final deliverables will be at least three of these qubricks, one for each team member, with accompanying plots and metrics demonstrating the algorithms' performance.

Project Plan

After getting up to speed with Workbench, studying the prospective algorithms more closely, setting up initial GitHub repositories, and selecting algorithms to implement in the first two weeks, we must begin setting up a continuous integration scheme for updating and testing our algorithms as we proceed in the coming weeks. We will also need to leave plenty of time at the end both for testing, analyzing resource usage, and optimizing the algorithms before putting together the final presentations. Here is a prospective, preliminary timeline for us to follow.

Week 1: Familiarize ourselves with Workbench via tutorial katas. Set up GitHub repos. Begin familiarizing ourselves with the prospective algorithms by reviewing the supplied paper.

Week 2. Finish familiarization with Workbench. Select algorithms to implement, recommended adders either in-place or out-of-place, per person. Begin writing implementations of routines in Workbench. Begin setting up continuous integration.

Week 3. Continue writing implementations of algorithms in Workbench. Change algorithms if needed. Finish setting up continuous integration, including writing tests.

Week 4: Move towards finishing up writing implementations of algorithms in Workbench. First studies of resource usage. Develop a pipeline for generating, styling, and storing plots and analysis, including iterating on visual clarity and other points of presentation and visual communication.

Week 5: Finish writing implementations of algorithms in Workbench. Continue analyzing resource usage. Move towards finishing a pipeline for generating, styling, and storing plots.

Week 6: Continue analyzing resource usage. Begin implementing optimizations as needed. Finish pipeline for generating, styling, and storing plots. Write and submit the midterm report.

Week 7: Continue analyzing resource usage and implementing optimizations as needed.

Week 8: Move towards finishing analyzing resource usage and implementing optimizations as needed. Begin putting together a poster for presentations using our plots and metrics.

Week 9: Finish analyzing resource usage and implementing optimizations as needed. Finish poster for presentations. Submit a PDF of the poster and begin printing for the ENGINE capstone presentation. Begin writing the final report.

Week 10: Present printed poster at ENGINE capstone session. Finish writing and submit the final report.

Each week will also have a weekly report to write and submit NLT Thursday mornings, meetings with the industry mentor Monday mornings, and meetings with the instructor Thursday mornings.

Gantt chart for preliminary project timeline with completion statuses filled in with a darker shade of blue

Team member assignment is dictated by the implementation of all these steps for their chosen algorithm. We have chosen the following algorithms to begin trying to implement:

Kush – TTK in-place ripple carry adder
Thomas – CT out-of-place ripple carry adder
Chris – Wang out-of-place ripple carry adder

All team members will work on the midterm report, poster, and final report.

One key risk is that the chosen algorithm may be too difficult to implement within the allotted time, especially under the constraints of Workbench. Some algorithms that are efficient in theory may be difficult to express within the framework or may introduce additional overhead due to ancilla management, control logic, or register structure. In addition, meaningful performance differences often only emerge at larger input sizes, while Workbench and our available computational

resources limit us to relatively small-scale simulations. This makes it difficult to directly observe asymptotic behavior or identify tipping points.

To mitigate these risks, we will begin implementation early (starting this week) to assess feasibility and switch to a simpler alternative if necessary before significant time is lost. We will also prioritize algorithms that are both theoretically meaningful and practical to implement in Workbench, and use scaling trends within the accessible range to infer behavior at larger problem sizes.